Take-Home Assignment: Agentic AIfor Discharge Summaries
Role: AI Engineer Time: Up to one full working day. We expect ~6?10 focused hours; you have a
48-hour window from receipt to submit, so you can pick your day. AI assistants: Allowed and
encouraged ? but you must understand and be able to explain every line of what you submit.

Overview
You will build an agentic AI system that reads a patient's raw source notes and produces a
structured, clinically safe discharge summary draft for clinician review. Source documents are
messy and incomplete, exactly as in the real world ? so the system must plan, decide which
information it needs, recover when things are missing or contradictory, and know when not to act
on its own.
This is not a single LLM call. We are evaluating how you design an agent: planning, tool use,
robustness, control, and ? above all in a clinical setting ? its refusal to invent facts.
The assignment has two parts. Part 1 is the bar. Part 2 is a stretch that distinguishes the
strongest submissions. A strong Part 1 alone is a valid, complete submission.

The Data
You will receive a set of patients. For each patient you get a folder of source-note PDFs, which
may include:
Admission note
Progress notes (one or more)
Laboratory results
Medication records (admission and discharge)
The documents are intentionally realistic: some are verbose, some are incomplete, some contain
results that are still pending, some contain medication changes with no stated reason, and
some contain information that conflicts between notes. Reading and making sense of these PDFs is part of the task.
All data provided is synthetic. Do not upload it to any third-party service that would retain it,
and do not submit any real patient data of your own.

Part 1 ? The Discharge SummaryAgent (required)

Build an agent that, given a patient's source-note PDFs, produces a structured discharge-
summary draft for clinician review.

Required sections of the output
Patient demographics • admission & discharge dates • principal and secondary diagnoses •

hospital course • procedures • discharge medications with changes from admission clearly noted • allergies • follow-up instructions • pending results • discharge condition.

Hard requirements

1. A real agent loop. The system must plan and re-plan based on what it reads and what tools return ? not a fixed, hardcoded pipeline.
2. PDF ingestion. Read and extract the relevant content from the source PDFs yourself.
3. No fabrication ? the core guardrail. The agent must never invent or guess a clinical fact.

Any required field it cannot source from the documents must be explicitly marked missing or pending and flagged for clinician review. The output is always a draft for review, never an auto-finalized clinical document.

4. Handle pending / missing data. If a lab is pending or a note is absent, say so ? do not fill in a plausible value.
5. Medication reconciliation. Compare admission vs. discharge medications and surface changes. If a medication was added, stopped, or changed with no documented reason, flag it for reconciliation rather than silently resolving it.
6. Handle conflicting information. If two notes disagree (e.g., different discharge diagnoses), flag the conflict ? do not arbitrarily pick one.
7. Use tools, and decide when. You may mock external tools where needed (for example a drug-interaction lookup and an escalation/"flag for clinician review" action). The agent must decide when to call them. If an interaction or safety concern is found, it must be surfaced and escalated, not buried.
8. Robust failure handling. Tools and document reads can fail, time out, or return empty. The agent must retry, fall back, or report ? never crash, and never behave as if a failed call succeeded.
9. Control. Enforce a hard step/iteration cap so the agent cannot run forever.
10. Observability. Emit a readable trace for each step: reasoning ? tool/action chosen ? inputs ? result ? next decision.

Part 2 ? Learning from Doctor Edits (stretch)

In production, a clinician edits the agent’s draft before finalizing. Those edits are signal: the system should learn from them so future drafts need fewer corrections and become more accurate over time.

We do not have real doctor-edited data ? so part of the challenge is manufacturing the feedback loop yourself and proving it works.

You must:

1. Define a reward / accuracy signal derived from edits ? e.g., section-level match rate or normalized edit distance between the agent’s draft and the corrected version (less editing = higher reward).
2. Create a way to generate edit signal. Build a simulated reviewer: a stand-in "doctor"

that
applies a consistent (and, to the agent, hidden) editing policy to drafts, producing
(draft,
edited) pairs. This lets you train and measure without real clinicians in the loop.
3. Implement a learning mechanism that uses accumulated edits to improve future drafts.
Choose your approach and justify it ? for example a contextual bandit over
prompt/template strategies rewarded by reduced edit burden, preference fine-tuning
(DPO/SFT) on the pairs, a learned reward model with best-of-N selection, or a structured
correction-memory injected into future prompts.
4. Show measurable improvement. Report a before/after metric (e.g., average edit
distance or
section accuracy) on a held-out set, with an improvement curve over iterations.
5. Discuss the limitations. Address the cold-start / limited-data problem, and the risk
that
optimizing to reduce edits can be gamed ? an agent can lower edit distance by becoming
vaguer or mimicking style rather than getting the medicine right. Explain how you keep
the
learning loop from degrading the Part 1 safety guarantees.
A clean, well-measured method that demonstrably lowers edit burden beats a sophisticated
method that doesn't run or isn't evaluated.

Required: Working Video Demo
Record a 3?5 minute screen recording (e.g., Loom) in which you:
Run the agent live on at least two patients, including one with missing/pending data or
a
conflict.
Walk through a step trace and point out a moment where the agent chose to flag or
escalate
rather than guess.
If you attempted Part 2, show the before/after improvement.
The video is required and doubles as proof the system runs.

What to Submit
1. Source code in a repo (GitHub link or zip) with clear run instructions.
2. Generated discharge-summary drafts and step traces for every patient in the provided
set.
3. For Part 2 (if attempted): the simulated reviewer, the learning mechanism, and the
before/after metric + curve.
4. The video demo (link).

5. A short README (1?2 pages) covering: your agent's loop design, how you enforce the
no-
fabrication guardrail, how you handle failures and conflicts, your Part 2 reward design
and

results, the limitations of your approach, and what you would do with more time.

Rules & Constraints
Time-box to roughly one working day; submit within the 48-hour window.
Any LLM provider is fine. Keep API keys out of the repo.
AI coding assistants are allowed; you must be able to explain everything you submit.
Frameworks (LangGraph, CrewAI, etc.) are permitted, but a clear from-scratch agent loop

is slightly preferred and you should be able to justify every decision the agent makes.
Do not use or submit any real patient data.
Partial work is fine. If you run out of time, say clearly in the README what is done, what
isn’t, and what you’d do next.

What We’re Evaluating
Clinical safety (never fabricating, always flagging the unknown) above all; the quality of your
agentic design (planning, tool use, recovery, control); how you handle the messy and
contradictory cases; the soundness of your Part 2 learning loop and its measured results; and the
clarity of your code, traces, and reasoning.
Good luck ? we’re more interested in sound judgment under uncertainty than in a polished
happy path.